

SIMATS ENGINEERING

Assessment Tool 2 — Constraint-Based Problem Solving (CSA17: Artificial Intelligence)

Model Answers — Annexure A

Question 1 — Doctor Shift Scheduling (Backtracking Search)

Problem restated: Assign doctors D1, D2, D3 to shifts {Morning, Afternoon, Night}, one doctor per shift, subject to:

- C1: $D1 \neq \text{Night}$
- C2: D2 must occupy an earlier shift than D3 (Morning < Afternoon < Night)
- C3: All-Different — only one doctor per shift
- C4: $D3 \neq \text{Morning}$
- C5: Every doctor gets exactly one shift

Variable order used: D1, D2, D3. Value order tried for each variable: Morning → Afternoon → Night (standard backtracking with constraint checking after every assignment).

Step-by-step backtracking trace — Branch A (D1 = Morning)

Step	Assignment attempted	Constraint check	Result
1	D1 = Morning	$D1 \neq \text{Night} \rightarrow$ satisfied	Assign & continue
2	D2 = Morning	All-Different (D1=Morning) $\rightarrow$ violated	Reject, try next value
3	D2 = Afternoon	All-Different $\rightarrow$ satisfied (no conflict with D1)	Assign & continue
4	D3 = Morning	$D3 \neq \text{Morning} \rightarrow$ violated	Reject, try next value
5	D3 = Afternoon	All-Different (D2=Afternoon) $\rightarrow$ violated	Reject, try next value
6	D3 = Night	All-Different $\rightarrow$ satisfied; $D3 \neq \text{Morning} \rightarrow$ satisfied; $D2 < D3$ (Afternoon < Night) $\rightarrow$ satisfied	Assign — all variables assigned, all constraints satisfied $\rightarrow$ GOAL

Alternative branch — Branch B (D1 = Afternoon)

For completeness, backtracking also explores the sibling branch where D1 takes the next value in its domain, showing that a second valid schedule exists:

Step	Assignment attempted	Constraint check	Result
1	D1 = Afternoon	$D1 \neq \text{Night} \rightarrow$ satisfied	Assign & continue
2	D2 = Morning	All-Different $\rightarrow$ satisfied	Assign & continue
3	D3 = Morning	$D3 \neq \text{Morning} \rightarrow$ violated	Reject
4	D3 = Afternoon	All-Different (D1=Afternoon) $\rightarrow$ violated	Reject
5	D3 = Night	All-Different $\rightarrow$ satisfied; $D3 \neq \text{Morning} \rightarrow$ satisfied; $D2 < D3$ (Morning < Night) $\rightarrow$ satisfied	Assign — GOAL (second valid schedule)

Final valid schedule(s)

Backtracking search terminates successfully as soon as a complete, consistent assignment is found. The first goal reached (Branch A) is:

- D1 $\rightarrow$ Morning
- D2 $\rightarrow$ Afternoon
- D3 $\rightarrow$ Night

A second, equally valid schedule (Branch B) is:

- D1 → Afternoon
- D2 → Morning
- D3 → Night

Observation: In every valid solution D3 is forced into the Night shift. This is because C4 rules out Morning for D3, and if D3 took Afternoon then D2 would need Morning (to satisfy C2), leaving D1 with Night — which violates C1. Hence D3 = Night is the only value consistent with all constraints, and D1, D2 can freely take Morning/Afternoon in either order.

Question 2 — Robot Path Planning on a 5×5 Grid

Note on approach: The question asks for BFS while also specifying a priority queue and a Manhattan-distance heuristic. Plain BFS uses a FIFO queue and ignores heuristics, so it cannot use either of those two things. The heuristic + priority-queue requirement is the defining feature of A* Search (which generalises BFS's systematic layer-by-layer expansion by ordering the frontier with $f(n) = g(n) + h(n)$ instead of pure insertion order). The trace below therefore applies A* Search, built on BFS's node-expansion principle, using the Manhattan distance as $h(n)$ and move-cost $g(n)$, since this is the search strategy that actually matches every constraint listed in the question.

Grid layout used

S = start (0,0); G = goal (4,4); # = obstacle; rows/columns are 0-indexed, row increasing downward.

S	.	.	.	.
.	#	#	.	.
.	.	.	#	.
.	#	.	#	.
.	.	.	.	G

Obstacles: (1,1), (1,2), (2,3), (3,1), (3,3).

Heuristic and cost function

$h(n)$ = Manhattan distance to goal = $|\text{row}(n) - 4| + |\text{col}(n) - 4|$. $g(n)$ = number of moves taken from S. $f(n) = g(n) + h(n)$. Each move (Up/Down/Left/Right) costs 1; obstacle cells are never added to the frontier.

Step-by-step node expansion (A* built on BFS frontier expansion)

Step	Node expanded	$g(n)$	$h(n)$	$f(n)$	Frontier action
1	(0,0) S	0	8	8	Expand → generate (0,1), (1,0)
2	(0,1)	1	7	8	Expand → generate (0,2); (1,1) blocked
3	(0,2)	2	6	8	Expand → generate (0,3); (1,2) blocked
4	(0,3)	3	5	8	Expand → generate (0,4), (1,3)
5	(0,4)	4	4	8	Expand → generate (1,4)
6	(1,4)	5	3	8	Expand → generate (2,4)
7	(2,4)	6	2	8	Expand → generate (3,4); (2,3) blocked
8	(3,4)	7	1	8	Expand → generate (4,4) = G
9	(4,4) G	8	0	8	Goal popped from priority queue → STOP

Priority queue (open list) after each expansion

Event	Priority queue contents (f value)
After expanding (0,0)	(0,1) f=8, (1,0) f=8
After expanding (0,1)	(1,0) f=8, (0,2) f=8
After expanding (0,2)	(1,0) f=8, (0,3) f=8
After expanding (0,3)	(1,0) f=8, (0,4) f=8, (1,3) f=8
After expanding (0,4)	(1,0) f=8, (1,3) f=8, (1,4) f=8
After expanding (1,4)	(1,0) f=8, (1,3) f=8, (2,4) f=8
After expanding (2,4)	(1,0) f=8, (1,3) f=8, (3,4) f=8
After expanding (3,4)	(1,0) f=8, (1,3) f=8, (4,4) f=8
After expanding (4,4)	GOAL — search terminates

At each tie in $f(n)$, the node with the lower $h(n)$ — i.e. closer to the goal by Manhattan distance — is expanded first; this is what steers the search along the right-then-down corridor and around the two obstacle clusters instead of wastefully exploring toward (1,0).

Optimal path and cost

Path found: $(0,0) \rightarrow (0,1) \rightarrow (0,2) \rightarrow (0,3) \rightarrow (0,4) \rightarrow (1,4) \rightarrow (2,4) \rightarrow (3,4) \rightarrow (4,4)$.

Total cost = 8 moves, which equals the straight-line Manhattan distance between S and G ($|4-0| + |4-0| = 8$). Since no path can cost less than the Manhattan distance and this path avoids every obstacle, it is provably optimal.

Question 3 — Autonomous Rescue Robot in a Partially Observable Environment

a) Effect of each constraint on decision-making

- 4-directional movement (Up/Down/Left/Right): limits the branching factor at every node to at most 4, keeping the search tree tractable but also meaning some geometrically 'shorter' diagonal paths are unavailable — the agent must plan in orthogonal steps.
- Blocked cells (obstacles): prune the state space — the agent must treat unexplored cells as potentially traversable until sensed, so obstacles discovered mid-path force re-planning rather than being known upfront.
- Limited sensing (adjacent cells only): the environment is partially observable, so the agent cannot compute a single optimal path in advance; it must interleave acting and sensing, updating its internal map as it moves (an online search problem).
- Variable move cost (1 normal, +2 in risky zones, so 3 total): turns the problem from a pure shortest-path search into a cost-sensitive search — the cheapest path in number of steps is not necessarily the cheapest in total cost, so the agent must weigh distance against risk exposure.
- Minimize total cost while staying safe: introduces a dual objective (cost vs. safety), which the agent must balance rather than optimizing distance alone.
- Dynamic knowledge updates (online search): the agent's model of the grid is incomplete and revised continuously, so any committed path can become invalid and must be recomputed — this rules out any purely offline, one-shot planning algorithm.

b) Solution approach: Online search using Uniform Cost Search (LRTA*-style)

Because the environment is only partially observable and costs vary (normal cells = 1, risky cells = 3), Uniform Cost Search (UCS) is the right base algorithm for cost-optimality, and it must be run in an online fashion since the full map isn't known in advance.

- Step 1 — Sense: at the current cell, observe the (up to 4) adjacent cells to learn which are free, blocked, or risky.
- Step 2 — Update model: incorporate newly sensed information into the agent's internal grid map.
- Step 3 — Expand frontier: from the current cell, generate reachable neighbours with their true known step cost (1 or 3); insert them into a priority queue ordered by cumulative path cost $g(n)$ (UCS uses $g(n)$ only, no heuristic).
- Step 4 — Select and move: pop the lowest-cost node from the priority queue and physically move the robot there (this is what makes it an 'online' agent rather than an offline planner — it commits to actions based on current knowledge).
- Step 5 — Repeat: re-sense from the new cell, update the map, and continue expanding/selecting until the survivor's location G is reached.
- Step 6 — Replan on surprise: if a previously assumed-free cell turns out to be blocked (or a risky zone is discovered), discard the invalidated portion of the plan and re-run the cost search from the current cell — this is the essence of an online search agent (e.g., LRTA*).

This produces a path that is cost-optimal with respect to everything the robot has sensed so far, while remaining safe, because risky cells are only entered when no cheaper safe alternative exists — the +2 penalty naturally biases UCS's priority queue away from them.

c) AI concepts demonstrated

- Intelligent agent: the rescue robot perceives its environment through adjacent-cell sensing (percepts) and acts through movement (actuators), continuously cycling perceive → decide → act, which is the definition of an agent.

- Rationality: the agent is rational because, given what it has perceived so far, it always selects the action expected to minimize total cost while avoiding unsafe outcomes — it does the best it can with incomplete information, which is the standard for bounded/limited-information rationality (as opposed to omniscience).
- Environment type: the rescue scenario is partially observable (only adjacent cells are sensed), dynamic-in-knowledge though physically static (the grid itself doesn't move, but the agent's knowledge of it changes), sequential (each move affects future options), stochastic from the agent's viewpoint (outcomes of exploring unseen cells are uncertain until sensed), and single-agent.

d) Justification for the chosen approach

Uniform Cost Search run as an online search agent is the most suitable choice for three reasons drawn directly from the constraints. First, UCS guarantees the cheapest-cost path for graphs with non-uniform, non-negative edge costs, which matches the 1-vs-3 cost structure exactly — a plain BFS would find the fewest-steps path but ignore the risky-zone penalty, and could return an unsafe, more expensive route. Second, since the environment is only partially observable, no purely offline algorithm (which needs the complete map upfront) can be applied at all — the search has to be interleaved with sensing, which is exactly what an online search agent provides. Third, an online agent naturally supports re-planning when new obstacles or risky zones are discovered, satisfying the requirement that the robot 'must update its knowledge dynamically.' Together, these make UCS-as-online-search strictly better suited here than offline BFS/DFS (cost-blind and map-dependent) or a heuristic method like A* (which would need a reliable heuristic over a map the robot doesn't fully have yet).